FPGA-Based Neural Network Accelerator

Design, Implementation, and Verification of a Verilog Multi-Layer Perceptron Inference Core

Project Report

Prepared by: ***Sumit Maheshwari***

Affiliation: ***Indore Institute of Science and Technology***

Date: ***12/6/26***

Repository: github.com/roboticist-blip/FPGA_Workshop

1. Abstract

This report describes the design, implementation, and verification of an inference accelerator for a small neural network, which is directly implemented in Verilog hardware description language for a Xilinx Artix class FPGA. In contrast to an MLP operating as software on a CPU/GPU, this design performs the MLP computation via the hardware (i.e. parallel multipliers, adder trees, registers, finite state machine) without any processor or instruction set.

A two-layer fully connected neural network (4 input neurons -> 4 hidden neurons -> 2 output neurons) has been implemented using fixed-point arithmetic and currently with all weights and biases programmed into ROM as part of the test process. This project has been simulated using Icarus Verilog, and the results were verified with hand-calculated expected results for multiple test vectors,s and all matched.

An effort has been made to design the architecture using parameterised, reusable modules so that the same components can be easily scaled up to accommodate bigger feedforward networks.

2. Introduction

2.1 What this project is

This project is a hardware implementation of a neural network accelerator, that is, a digital circuit, all of whose parts have been written in Verilog, performing the arithmetic of a neural network inference phase. It does not depend on any kind of OS, CPU instruction set, or framework like TensorFlow or PyTorch. Every multiplication, addition, and activation function is implemented as a real connection of logic gates and registers on the FPGA structure.

In more detail, the design is a Multi-Layer Perceptron (MLP), which is the simplest neural network that exists. It consists of a single hidden layer. In the MLP, the network receives as input a vector with four values and gives as output a vector with two values. Each node computes the weighted sum of its inputs, sums up a bias value and finally computes an activation function (ReLU).

2.2 Motivation and context

The project is inspired by the AI_Accel folder from another one-week FPGA workshop repository, where there were some hand-built building blocks, including combinational MAC, clocked accumulating multiplier, and one hardcoded 4-input “neuron”. The latter modules proved the basic arithmetic primitive (multiply-accumulate), but they did not have any generalisation or formation of an actual multi-layer network.

In this project, the idea is implemented further in the form of an actual multi-layer network using parameterised modules, a controller, requantization between layers, and a simulation – specifically designed for the reuse of those very building blocks by just changing width parameters.

2.3 Why implement a neural network in hardware?

Computation on software on the CPU or the GPU involves the execution of the instructions of the neural network one by one, where the weight is loaded from memory, multiplied, and accumulated in a loop. While this approach offers flexibility, whereby all processors support all networks, it consumes computational resources on instruction fetching, memory fetching, and looping overhead that have no connection with the computations themselves.

On the other hand, the hardware accelerator implements the network-specific computations using a combination of gates and flip-flops. The multiply-and-accumulate of each neuron is implemented using actual circuitry and computes in parallel at every clock cycle; there is no instruction fetch and no looping overhead involved. The same principle lies behind the purpose-designed Neural Processing Units (NPUs) in smartphones and in Tensor Processing Units (TPUs) used in the data centres: the purpose-designed circuits outperform the generic computation by far.

However, there is a tradeoff: the cost of modification of a hardware design after it has been fabricated (or even after the synthesis of FPGAs) is much higher than that of changing a line in software. It is for this very reason that the current project begins with a simple network that can be easily verified.

2.4 Project objectives

1. Develop a parametric hardware neuron that can calculate N-length dot product, bias summing, and apply the ReLU activation function, and can be used across multiple layers of various widths.
2. Combine several neurons to form a fully connected layer with concurrent processing.
3. Properly address the precision issue in the transition between two layers (requantization), preventing silent overflow.
4. Organise the multi-layer processing pipeline through an explicit finite state machine rather than an implicit timing assumption.
5. Check the functionality in simulations using expected outputs before synthesising the design in hardware.
6. Develop a design that can scale (with no changes in structure) to a wider network architecture.

3. Background and Theory

3.1 The artificial neuron

The neuron calculates the weighted sum of its inputs, applies a bias to it, and then sends the value through an activation function that is non-linear in nature:

$$y = f(\Sigma (x_i \times w_i) + b)$$

where: x_i = inputs, w_i = weights, b = bias, f = activation function

Each weight corresponds to the strength with which the respective input affects the output of the neuron; this is determined during the learning process (software). However, this assignment does not require the learning part; only the feed-forward phase is implemented here.

3.2 Why a non-linear activation function is necessary

In the absence of a non-linear activation function, there will be no mathematical sense in adding more than one neuron layer, as any combination of pure linear functions will remain just another linear function, so that the deep network becomes a network of one layer essentially. It is the activation function that provides a multi-layer network capability to model complicated relationships.

The activation function chosen for this design is called **ReLU (Rectified Linear Unit)**, the most popular activation function in today's neural networks:

```
ReLU(x) = max(0, x)
i.e. negative values become 0; non-negative values pass through unchanged.
```

The activation function used was ReLU since it is very easy to compute hardware-wise – simple compare with zero and pass-through depending on the result – unlike other activation functions such as sigmoid or tanh that require either a lookup table or an approximate hardware circuit.

3.3 Fixed-point arithmetic and why this design avoids floating point

Floating point operations (which come built in as a default operation within many software platforms) need significantly more hardware components, like specialised floating point operations units or complex soft-logic components, compared to fixed point operations with integer values. In the case of a compact edge inference accelerator, the cost incurred due to this is not worth much.

The design uses fixed-point signed integers across the board:

- The design uses fixed-point signed integers across the board:
- Inputs and Weights: 8-bit signed integers (INT8)
- Intermediate Accumulation (running sum operation in a neuron, before the activation function): 32-bit signed integer to ensure that the operations of the sum of multiple INT8 * INT8 operations do not exceed the range.

Intermediate values are transferred from one layer to another and saturated back to 8-bit values.

3.4 Requantization and saturation

Since each neuron's internal accumulator is larger in size (32-bit) than the network's external data format (8-bit), some sort of translation process is necessary between layers. Two methods can be employed to deal with the situation when the data value exceeds the capacity of the smaller data size:

- Truncation: just drop the higher-order bits. It's a no-cost operation, but it is highly risky because an overflowed data value will simply roll over and become a small and valid-looking but completely wrong number, hard to pinpoint.
- Saturation: clamp the data value to either the maximum or min representable value. It is the method that is implemented in this design. If an overflow happens, it becomes clearly visible because the value is clamped to its maximum, and it is easy to spot.

4. System Architecture

4.1 Network topology

The implemented network is a standard fully-connected feedforward MLP with one hidden layer:

```
Input vector (4 values, INT8)
      |
      v
 Hidden Layer  -- 4 neurons, each: y = ReLU( dot(x, w) + b ), 4-wide dot product
      |   (4 values, requantized 32-bit -> 8-bit, saturating)
      v
 Output Layer  -- 2 neurons, each: y = ReLU( dot(x, w) + b ), 4-wide dot product
      |
      v
Output vector (2 values)
```

This is equivalent mathematically to a Dense(4) -> Dense(2) model in software libraries like Keras/PyTorch, with the sole distinction being that in the present case, the computational graph is hardware-based as opposed to software-based.

4.2 Module hierarchy

Module	Role
neuron.v	Single neuron: parameterised N-wide dot product + bias + ReLU, with the result registered on the clock edge.
neuron_layer.v	Instantiates N_OUT neuron.v instances in parallel, all driven from the same shared input vector — implements one fully-connected layer.
requant.v	Saturating conversion of a layer's 32-bit accumulator outputs down to the 8-bit format that the next layer expects as input.
weights_l1.v / weights_l2.v	Hardcoded ROM supplying each layer's weights and biases. Hand-picked small integers for verification, not trained values.
fsm_ctrl.v	Finite state machine sequencing the pipeline (IDLE → LOAD_INPUT → LAYER1_WAIT → LAYER2_WAIT → DONE) and producing a one-cycle DONE pulse once the output is valid.
mlp_top.v	Top-level module: wires the ROMs, both layers, the requantizer, and the FSM together into the complete network.
tb_mlp_top.v	Testbench: drives test input vectors through the network and records/dumps the resulting outputs for verification.

4.3 Data flow

```
X_IN (4x8b)
  |
  v
[weights_l1 ROM] --> [neuron_layer: 4 neurons] --> l1_Y (4x32b, post-ReLU)
                                                     |
                                                     v
                                          [requant: 32b -> 8b, saturating]
                                                     |
                                                     v
                                                l1_Yq (4x8b)
                                                     |
[weights_l2 ROM] ----------------------> [neuron_layer: 2 neurons] --> Y_OUT (2x32b)
```

[fsm_ctrl] runs alongside, sequencing timing and raising DONE when Y_OUT is valid.

4.4 Pipeline timing

The dot product of each layer is pure combinational logic, with only the final ReLU'd output being latched in a register on the rising edge of the clock. It follows that the output is only valid for two rising edges of the clock after a new input vector has been provided – one for each layer. The reason why there is a need for the control FSM is precisely for this kind of purpose.

FSM State	Description	done
IDLE	Waiting for the start signal.	0
LOAD_INPUT	The input vector is presented to layer 1.	0
LAYER1_WAIT	Layer 1's registered outputs become valid on this edge.	0
LAYER2_WAIT	Layer 2's registered outputs become valid on this edge.	0
DONE	The output vector is valid and held stable.	1 (pulse)

5. Implementation Details

5.1 The parameterised neuron (neuron.v)

The fundamental building block used in this circuit design is a single parameterizable neuron unit, which consists of 3 parameters in Verilog that define the shape of the module:

Parameter	Meaning	Value used in this design
N	Number of inputs to the neuron	4
DW	Bit width of each input/weight element	8 (INT8)
ACC_W	Bit width of the internal accumulator/output	32 (INT32)

Inside the module, the generate loop in Verilog instantiates N parallel multipliers (not each multiply explicitly listed), adds them using an adder tree structure, adds the bias, applies ReLU, and then registers the result. Since the width of the network is completely defined by parameters, the exact same module is used unchanged for both the 4-wide hidden layer and 4-wide output layer in this design, and is meant to be used again by changing N for a larger network.

5.2 The fully-connected layer (neuron_layer.v)

The layer is constructed by creating N_OUT instances of neurons.v with each neuron having the same input vector and an individual part of the weight/bias bus. All the neurons operate concurrently without

any multiplexing or sharing of hardware between neurons in this design approach, at the cost of extra logic and DSP components.

5.3 Weight storage (weights_l1.v, weights_l2.v)

The weights and biases have currently been hard-coded into the Verilog code as integers of choice, such that all results generated can be verified manually. It must be noted that the weights and biases that have been hard-coded into the Verilog code are not training weights for a real model. Hard-coding makes weight loading not a parameter during the proving of the correctness of the core computational flow (datapath, requantization, and control FSM).

5.4 Design decisions and rationale

Decision	Rationale
Combinational dot product, registered only at the output	Avoids multi-cycle MAC sequencing complexity; well within timing closure for a network this small on an Artix-class device.
All neurons in a layer are computed in parallel	Maximises speed and simplifies control logic at the cost of additional multiplier/DSP usage — an acceptable tradeoff at this network size.
ReLU on every layer, including the output layer	Standard, hardware-cheap activation. Note: this constrains the network's output to be non-negative, a modelling constraint to keep in mind once real trained weights/tasks are used.
Saturating requantization between layers	Avoids the silent-wraparound failure mode of truncation; an overflow is visibly clipped rather than masquerading as a valid value.
Hardcoded ROM weights	Removes weight-loading as a variable while the compute pipeline itself is being verified; deliberately deferred, not an oversight.

6. Verification Methodology

Design verification took place using simulations, rather than using synthesis or programming as the initial step, as it is normally done for designing hardware, because simulations provide the benefit of knowing all internal signals and are much quicker compared to synthesis and programming.

6.1 Toolchain

- Simulator: Icarus Verilog (iverilog / vvp)
- Waveform viewer: GTKWave
- Target synthesis tool (subsequent stage): Xilinx Vivado, targeting an Artix-7 class device

6.2 Verification approach

1. Each test input vector was hand-calculated through the network's mathematics (dot product → bias → ReLU, for both layers) to produce an expected output independent of the simulator.
2. The same input vector was applied to the testbench, and the simulated output was recorded.
3. Hand-calculated and simulated results were compared directly for an exact match.

4. The waveform was additionally inspected to confirm the FSM's state sequence, and the DONE pulse timing matched the design's intended timing model (Section 4.4).

7. Results

This section presents the verification results.

7.1 Functional simulation results

The following table compares hand-calculated expected outputs against the values observed in simulation, for each test input vector applied to the network.

Input vector [x0,x1,x2,x3]	Expected [y0,y1]	Simulated [y0,y1]	Match
[2, 3, -1, 4]	[8, 8]	[8, 8]	✅
[0, 0, 0, 0]	[0, 0]	[0, 0]	✅
[5, 5, 5, 5]	[20, 16]	[20, 16]	✅

Table 7.1 — Hand-calculated vs. simulated output comparison.

7.2 Waveform analysis

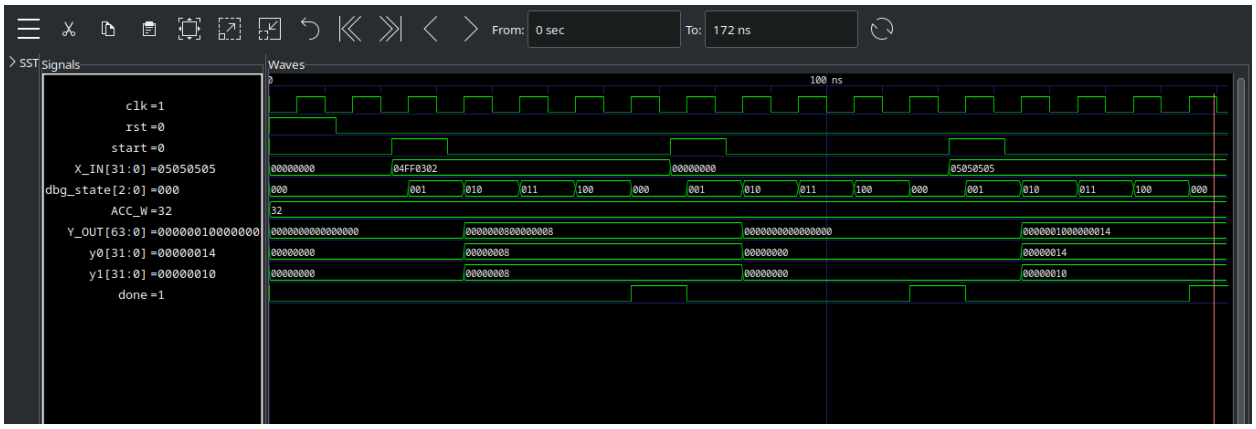

Figure 7.1 — Simulation waveform.

Observed behaviour to confirm in the waveform:

- dbg_state steps cleanly through its expected sequence (IDLE → LOAD_INPUT → LAYER1_WAIT → LAYER2_WAIT → DONE → IDLE) for each test vector.
- y0 / y1 settle to their final values no later than the LAYER2_WAIT state, and remain stable through DONE.
- Done asserts for exactly one clock cycle, aligned with the cycle in which Y_OUT is guaranteed valid.
- No unexpected glitches or metastable-looking transitions appear on any signal between state changes.

7.3 Synthesis/resource utilisation (Vivado)

The results we got after synthesising this design in Vivado:

Resource	Utilization	Available	Utilization %
LUT	207	32600	0.63
FF	105	65200	0.16
IO	103	210	49.05

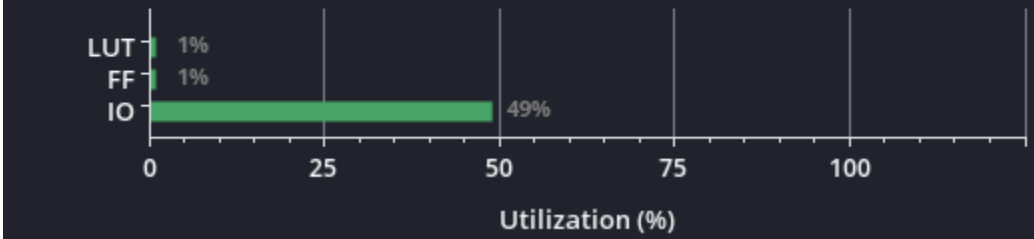

Figure 7.2 — Vivado synthesis resource utilisation.

7.4 Design Timings

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	1.875 ns	Worst Hold Slack (WHS)	0.210 ns	Worst Pulse Width Slack (WPWS)	4.500 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	50	Total Number of Endpoints	50	Total Number of Endpoints	106

All user specified timing constraints are met.

Figure 7.3 — Vivado synthesis Design Timing Summary.

7.5 Power estimate

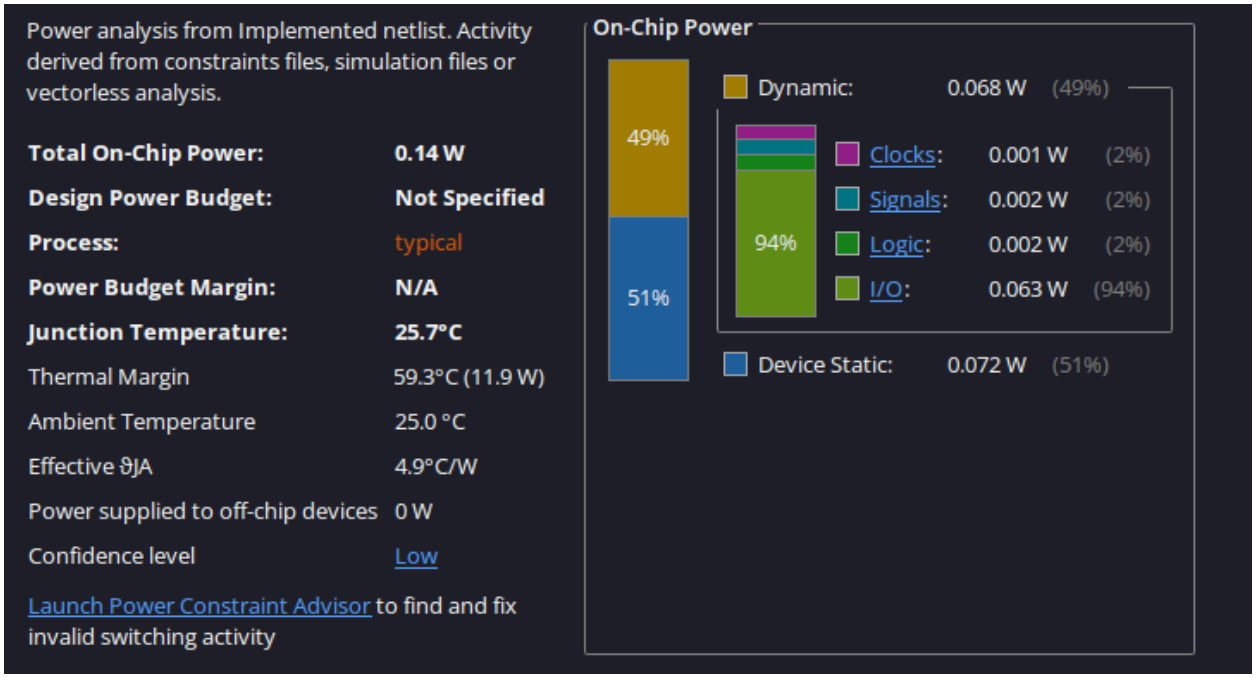

Figure 7.4 — Vivado synthesis Power Analysis Summary.

The rest of the result reports are available at:
https://github.com/roboticist-blip/FPGA_Workshop/tree/main/AI_Accel/Neuron/Reports

8. Discussion

The simulation proves the correctness of the implementation of the desired MLP's feed-forward pass by the described core structure, which includes a parameterised neuron, a parallel fully-connected layer, saturation requantization between layers, and a control FSM. All functions provided the exact result of calculations, which were done manually. The timing of operations is two clock edges per inference, and one-cycle completion indication was verified in the waveform.

8.1 Known limitations of the current design

- Weights and biases are hard-coded at synthesis time; the design does not allow for a different trained model to be loaded without resynthesizing.
- ReLU is used on the output layer, ensuring that the outputs are all positive; this works fine for this network, but it should be considered a true limitation for networks that require negative outputs.
- All neurons in one layer are calculated in parallel; however, this approach does not scale up to large layers without re-thinking the trade-offs between parallel and sequential computing.

9. Conclusion

The main success of this project has been the design, implementation, and verification of a small, parameterisable neural network inference accelerator written in Verilog. This design is not an end in itself

but has been designed to lay a foundation upon which a bigger design can be based. The design has been deliberately made small and fully hand-verifiable to make sure that all the building blocks used in it are trustworthy.

The project has been successfully verified, with functional simulation results matching all hand-calculated test vectors. Synthesis on the Xilinx Artix-7 device confirms the design is resource-efficient and meets all timing requirements. Having validated the core building blocks and the control pipeline, the architecture is now ready to be scaled to support larger, real-world network topologies.

10. Future Work

1. Scale the same building blocks (neuron.v, neuron_layer.v, requant.v) up to a larger, real-world feedforward network topology.
2. Replace hardcoded ROM weights with runtime-loaded weights (via UART or AXI) once a real trained model is targeted.

Appendix A: Repository and File Reference

All source files referenced in this report are available in the project repository:

https://github.com/roboticist-blip/FPGA_Workshop/tree/main/AI_Accel

File	Description
neuron.v	Parameterised neuron: N-wide dot product, bias, ReLU.
neuron_layer.v	Parallel fully-connected layer wrapper.
requant.v	Saturating 32-bit → 8-bit requantizer between layers.
weights_l1.v / weights_l2.v	Hardcoded weight/bias ROMs for each layer.
fsm_ctrl.v	Pipeline control finite state machine.
mlp_top.v	Top-level integration module.
tb_mlp_top.v	Verification testbench.
mlp_top.vcd	Recorded simulation waveform.

Appendix B: How to Reproduce These Results

```
# Compile
iverilog -o mlp_sim neuron.v neuron_layer.v requant.v \
    weights_l1.v weights_l2.v fsm_ctrl.v mlp_top.v tb_mlp_top.v

# Simulate
vvp mlp_sim
```

```
# View waveform
gtkwave mlp_top.vcd
```

Within the waveform window, ensure that the value of dbg_state passes through the entire sequence and that the values of y0/y1 have stabilised before the signal done becomes high.